

CO2-Assesment Tool-02

Name: R. Guruprasad Achari

Reg no:192472094

Course Code: Csa1214

Course: Computer architecture

1. Debugging 2's Complement Overflow

Example: +120 + +50

Step 1: Convert to binary

- +120 = **01111000**
- +50 = **00110010**

Step 2: Add

```
01111000
+ 00110010
-----
10101010
```

Step 3: Result = 10101010 = **-86** (incorrect)

Issue: Overflow occurred because the actual result (**170**) is outside the 8-bit range (-128 to +127).

Solution: Detect overflow by checking if two positive numbers produce a negative result (or two negative numbers produce a positive result).

2. Debugging Ripple Carry Adder

Issue: Carry propagates through each bit one after another, causing delay.

Example:

- Bit 0 → Carry to Bit 1
- Bit 1 → Carry to Bit 2
- Bit 2 → Carry to Bit 3

Problem: Each bit waits for the previous carry.

Solution: Replace with **Carry Look-Ahead Adder (CLA)**.

Benefit: CLA computes all carries in parallel, reducing delay and improving processor speed.

3. Debugging Booth's Algorithm

Issue: Incorrect result for negative numbers.

Check:

1. Verify **Q0 and Q-1** bit pair.
2. Perform the correct **Add/Subtract** operation.
3. Apply **Arithmetic Right Shift**.
4. Ensure **sign extension** is maintained.

Possible Error: Wrong bit-pair checking or logical shift instead of arithmetic shift.

Solution: Use correct bit-pair rules and arithmetic right shift to preserve the sign bit.

4. Debugging Restoring Division

Issue: Extra restoration after every negative remainder increases execution time.

Workflow:

1. Shift left.
2. Subtract divisor.
3. If negative → Restore.
4. Repeat.

Problem: Frequent restoration operations.

Solution: Use **Non-Restoring Division**, which delays restoration until the end.

Benefit: Fewer operations, faster execution, and better efficiency.

5. Debugging IEEE 754 Floating Point

Issue: Adding very small and very large numbers gives inaccurate results.

Causes:

1. Exponent alignment shifts small number.
2. Normalization changes the result.
3. Rounding removes least significant bits.

Solution:

- Use **double precision (64-bit)** instead of single precision.
- Improve rounding methods.
- Rearrange calculations (add similar-sized numbers first).

Benefit: Higher accuracy and reduced precision loss.